



12. 연관 컨테이너와 컨테이너 어댑터

Heesung Oh

<https://github.com/3dapi>



- 키를 기준으로 저장하고 탐색하는 연관 컨테이너
- 정렬 연관 컨테이너와 비 정렬 연관 컨테이너
 - ◆ `std::set`, `std::multiset`
 - ◆ `std::map`, `std::multimap`
 - ◆ `std::unordered_set`, `std::unordered_map`
 - ◆ `std::stack`, `std::queue`, `std::priority_queue`
- 사용자 정의 타입의 비교와 해시
- 컨테이너와 객체 수명
- 반복자·포인터·참조의 유효성
- 컨테이너 선택 기준





- `std::set<Key>`: 중복되지 않는 키 저장
- `std::multiset<Key>`: 동등한 키를 여러 개 저장
- 비교 규칙에 따라 키 정렬
- 삽입, 탐색, 제거는 로그 시간 복잡도

```
#include <set>
std::set<int> uniqueness{30, 10, 20, 10};
std::multiset<int> scores{80, 90, 80, 70};
```

uniqueness → 10 20 30

scores → 70 80 80 90





- 기본 비교 함수 → `std::less<Key>`
- 비교 결과에 따라 키 자동 정렬
- 두 방향 비교가 모두 거짓 → 동등한 키
- 정렬 연관 컨테이너의 동등성
→ `operator==`가 아닌 비교 규칙 기준

```
// left도 앞서지 않고, right도 앞서지 않음  
!comp(left, right) && !comp(right, left)
```





- `set::insert()`는 삽입 위치와 성공 여부 반환
- 같은 키가 있으면 새 원소를 삽입하지 않음
- `multiset`은 중복 키를 모두 저장

```
std::set<int> ids;  
auto [position, inserted] = ids.insert(1001);  
if (inserted)  
{  
    std::cout << "inserted: " << *position << '\n';  
}
```

```
std::multiset<int> scores;
```

```
scores.insert(80);  
scores.insert(80);  
scores.insert(90);
```

요구	선택
키 중복 제거	<code>std::set</code>
같은 키 여러 개 저장	<code>std::multiset</code>



12.1.3 삽입, 탐색, 범위 검색과 제거

- `insert()`, `emplace()` → 원소 삽입
- `find()` → 키 탐색
- `contains()` → 존재 여부 확인
- `count()` → 동등한 키 개수
- `lower_bound()` → 키보다 작지 않은 첫 위치
- `upper_bound()` → 키보다 큰 첫 위치
- `equal_range()` → 동등한 키 범위
- `erase()` → 키 또는 반복자로 제거

```
std::set<std::string> names;

names.insert("Alice");
names.emplace("Bob");

auto position = names.find("Alice");

if (position != names.end())
{
    std::cout << *position << '\n';
}
```

```
std::set<int> values{10, 20, 30, 40, 50};
auto first = values.lower_bound(20);
auto last = values.upper_bound(40);
```



- 기본 오름차순 → `std::less<Key>`
- 내림차순 → `std::greater<Key>`
- 사용자 정의 타입 → 비교 함수 객체 지정 가능
- 비교 함수 → 엄격 약순서(strict weak ordering) 만족 필요
- 동점 발생 → 보조 기준으로 키 구분 필요

```
struct Player
```

```
{  
    int id;  
    int score;  
};
```

```
struct ComparePlayer
```

```
{  
    bool operator()(const Player& left, const Player& right) const  
    {  
        if (left.score != right.score)  
        {  
            return left.score > right.score;  
        }  
  
        return left.id < right.id;  
    }  
};
```

```
std::set<Player, ComparePlayer> ranking;
```



- `std::set`의 저장 원소 → 직접 수정 불가
- 키 변경 → 정렬 위치 변경 가능
- 일반적인 변경 → 제거 후 재삽입
- C++17 노드 핸들 → 노드 분리 후 키 수정 가능

```
std::set<int> values{10, 20, 30};  
auto position = values.find(20);  
// *position = 25; // 오류  
// 일반적인 변경  
values.erase(20);  
values.insert(25);
```

```
// C++17 노드 핸들  
std::set<std::string> names{"Alice", "Bob", "Carol"};  
  
auto node = names.extract("Bob");  
if (!node.empty())  
{  
    node.value() = "Bobby";  
    names.insert(std::move(node));  
}
```




12.2 std::map과 std::multimap

- `std::map<Key, T>` → 하나의 키에 하나의 값 연결
- `std::multimap<Key, T>` → 같은 키에 여러 값 연결
- 키 기준 자동 정렬
- 삽입·탐색·제거 → $O(\log n)$

```
#include <map>
#include <string>

std::map<int, std::string> itemNames{
    {1001, "Potion"},
    {1002, "Ether"}
};

std::multimap<std::string, std::string> categoryItems{
    {"Potion", "Small Potion"},
    {"Potion", "Large Potion"},
    {"Weapon", "Sword"}
};
```





- `value_type` → `std::pair<const Key, T>`
- `first` → 키
- `second` → 값
- 저장된 키 → 직접 수정 불가
- 저장된 값 → 수정 가능

```
for (const auto& entry : itemNames)
{
    std::cout << entry.first
                << ": "
                << entry.second
                << '\n';
}
```

```
auto position = itemNames.find(1001);
if (position != itemNames.end())
{
    position->second = "High Potion";
}
```



- operator [] → 값 참조 반환
- 키가 없으면 → 새 원소 삽입 후 값 초기화
- at() → 값 참조 반환
- 키가 없으면 → `std::out_of_range` 예외
- `const std::map` → `at()` 사용 가능, `operator []` 사용 불가
- `std::multimap` → `operator []`, `at()` 제공 안 함

```
// operator[]  
std::map<std::string, int> counts;  
++counts["Potion"];
```

```
// at()  
if (counts.contains("Potion"))  
{  
    std::cout << counts.at("Potion") << '\n';  
}
```



12.2.3 insert, emplace와 try_emplace

- insert() → 완성된 원소 삽입
- emplace() → 전달한 인수로 원소 생성
- try_emplace() → 키가 없을 때만 값 생성
- 값 생성 비용이 크거나 이동 전용 값 → try_emplace() 활용

```
std::map<int, std::string> names;
```

```
auto [position, inserted] = names.insert({1001, "Potion"});
```

```
names.emplace(1002, "Ether");
```

```
names.try_emplace(1003, "Elixir");
```

```
struct Item
```

```
{
```

```
    std::string name;
```

```
    int price;
```

```
    Item(std::string name, int price)
```

```
        : name(std::move(name)), price(price)
```

```
    {
```

```
    }
```

```
};
```

```
std::map<int, Item> items;
```

```
items.try_emplace(1001, "Potion", 50);
```



12.2.4 insert_or_assign과 값 갱신

- 키가 없으면 삽입
- 키가 있으면 함수에 따라 기존 값 유지 또는 갱신

함수	키가 없을 때	키가 있을 때
<code>insert()</code>	삽입	기존 원소 유지
<code>emplace()</code>	삽입	기존 원소 유지
<code>try_emplace()</code>	값 생성 후 삽입	값 생성 없이 유지
<code>insert_or_assign()</code>	삽입	기존 값에 대입
<code>operator[]</code>	기본값 삽입	값 참조 반환
<code>at()</code>	예외	값 참조 반환

```
std::map<int, std::string> names;  
names.insert_or_assign(1001, "Potion");  
names.insert_or_assign(1001, "High Potion");
```

```
std::map<std::string, int> counts;  
for (const std::string& name : itemNames)  
{  
    ++counts[name];  
}
```



- 구조적 바인딩 → 키와 값을 각각 이름으로 분리
- `const auto&` → 읽기 전용 순회
- `auto&` → 값 수정 가능
- 키 → `const`이므로 수정 불가

```
for (const auto& [id, name] : itemNames)
{
    std::cout << id << ": " << name << '\n';
}
```

```
std::map<int, int> prices{
    {1001, 50},
    {1002, 80}
};
for (auto& [id, price] : prices)
{
    price += 10;
}
```

- `id`는 `const int&`
- `price`는 `int&`
- `multimap`의 같은 키 범위 → `equal_range()` 사용

```
auto [first, last] = items.equal_range("Potion");
```



- 키의 해시값으로 버킷 선택
- 키 정렬 없음
- 순회 순서에 의미 없음
- 정확한 키 검색 중심
- 평균 탐색 $\rightarrow O(1)$
- 최악 탐색 $\rightarrow O(n)$

키
↓ hash
해시값
↓
버킷 선택
↓
동등 비교
↓
원소 탐색

컨테이너	중복 키
<code>unordered_set</code>	허용하지 않음
<code>unordered_multiset</code>	허용
<code>unordered_map</code>	허용하지 않음
<code>unordered_multimap</code>	허용



12.3.1 unordered_set, unordered_multiset

- unordered_set → 중복 키 허용 안 함
- unordered_multiset → 중복 키 허용
- 삽입 순서나 정렬 순서 보장 안 함

```
#include <unordered_set>
std::unordered_set<int> ids{30, 10, 20, 10};
std::unordered_multiset<int> scores{80, 90, 80, 70};

ids.insert(1001);
if (ids.contains(1001))
{
    ids.erase(1001);
}
```





12.3.2 unordered_map, unordered_multimap

- unordered_map → 하나의 키에 하나의 값
- unordered_multimap → 같은 키에 여러 값
- unordered_map → operator[], at(), try_emplace() 사용 가능
- unordered_multimap → operator[], at() 제공 안 함

```
std::unordered_map<int, std::string> itemNames;
```

```
itemNames[1003] = "Elixir";
```

```
itemNames.try_emplace(1004, "Antidote");
```

```
itemNames.insert_or_assign(1003, "High Elixir");
```





12.3.3 해시 함수, 동등 비교와 버킷

- 기본 해시 함수 → `std::hash<Key>`
- 기본 동등 비교 → `std::equal_to<Key>`
- 해시 함수 → 후보 버킷 선택
- 동등 비교 → 버킷 안에서 정확한 키 확인
- 버킷 번호 → 일반적인 코드에서는 직접 관리하지 않음

```
std::size_t hashValue = std::hash<std::string>{}("Potion");
```

```
using Table = std::unordered_map<  
    std::string  
    , int  
    , std::hash<std::string>  
    , std::equal_to<std::string>  
    >;
```

```
std::unordered_set<std::string> names{  
    "Alice", "Bob", "Carol"  
};  
std::cout << names.bucket_count() << '\n';  
std::cout << names.bucket("Alice") << '\n';
```



12.3.4 충돌, 적재율과 재 해시

- 서로 다른 키가 같은 버킷에 배치 → 해시 충돌
- 충돌 → 오류 아님
- 버킷 안의 동등 비교로 키 구분
- 적재율 → 원소 수 / 버킷 수
- `reserve(n)` → 원소 수 기준 저장 공간 준비
- `rehash(n)` → 버킷 수 기준 재구성 요청
- 재 해시 발생 → 모든 반복자 무효화
- 재 해시 발생 → 기존 원소의 포인터와 참조는 유지

버킷 0 :

버킷 1 : Alice → Carol

버킷 2 : Bob

버킷 3 :

```
std::unordered_map<int, std::string> items;  
items.reserve(1000);
```



- 사용자 정의 키 → 해시 함수와 동등 비교 필요

```
struct ItemKey
{
    int category;
    int id;
    bool operator==(const ItemKey& const) = default;
};

struct ItemKeyHash
{
    std::size_t operator()(const ItemKey& key) const noexcept
    {
        std::size_t first = std::hash<int>{}(key.category);
        std::size_t second = std::hash<int>{}(key.id);
        return first ^ (second << 1);
    }
};
```

// 동등한 키 → 반드시 같은 해시값

```
std::unordered_map<ItemKey, std::string, ItemKeyHash>
itemNames;
```

// 서로 다른 키 → 같은 해시값 가능

left == right → hash(left) == hash(right)



- 정렬 연관 컨테이너 → 비교 규칙 중심
- 비 정렬 연관 컨테이너 → 해시와 동등 비교 중심
- 평균 탐색 시간만으로 선택하지 않음
- 순회 순서·범위 검색·최악 비용·메모리 구조 함께 판단





- set, map 계열 → 비교 기준에 따른 정렬 순회
- unordered 계열 → 순회 순서 지정되지 않음

```
// 순회 키: 10, 20, 30
```

```
std::map<int, std::string> ordered{  
    {30, "C"},  
    {10, "A"},  
    {20, "B"}  
};
```

```
// 순회 순서 지정 없음
```

```
std::unordered_map<int, std::string> unordered{  
    {30, "C"},  
    {10, "A"},  
    {20, "B"}  
};
```

요구	적합한 범주
정렬 순회	set, map 계열
정확한 키 검색 중심	unordered 계열



- unordered 계열 → 해시 분포가 양호할 때 평균 상수 시간
- 충돌 집중 → 선형 탐색에 가까워질 수 있음
- 원소 수가 적으면 → 메모리 접근·할당 비용 영향 증가

범주	평균 탐색	최악 탐색
set, map 계열	$O(\log n)$	$O(\log n)$
unordered 계열	$O(1)$	$O(n)$





- 정렬 연관 컨테이너
 - 일반적으로 노드 기반 트리 구조
- 비 정렬 연관 컨테이너
 - 버킷 배열 + 원소 저장 구조
- 두 범주 모두
 - `std::vector` 같은 전체 연속 메모리 보장 없음
- 실제 성능
 - 메모리 접근 패턴의 영향도 고려



- 정렬 연관 컨테이너 → `lower_bound()`, `upper_bound()` 지원
- 정렬 연관 컨테이너 → 최소값·최대값·구간 검색에 적합
- 비 정렬 연관 컨테이너 → 정확한 키 검색 중심

```
std::map<int, std::string> players;  
auto first = players.lower_bound(1000);  
auto last = players.upper_bound(1999);
```

기준	set, map 계열	unordered 계열
키 배치	비교 함수 순서	해시 버킷
순회 순서	정렬됨	지정되지 않음
범위 검색	지원	지원하지 않음
사용자 정의 키	비교 규칙	해시 + 동등 비교
삽입 시 반복자	기존 위치 유지	재 해시 시 전체 무효



- 컨테이너 어댑터
 - ◆ 이미 존재하는 기본 컨테이너 (예: vector, deque, list)의 인터페이스를 제한하거나 변경하여, 특정 목적에 맞는 새로운 형태의 자료 구조로 감싸서 제공하는 클래스 템플릿
- std::stack → 후입선출 (LIFO)
- 기본 기반 컨테이너 → std::deque





- 마지막에 넣은 원소 → 가장 먼저 처리
- `push()` → 원소 추가
- `top()` → 가장 위 원소 확인
- `pop()` → 가장 위 원소 제거
- `pop()` → 제거한 값 반환 안 함
- 빈 상태에서 `top()`, `pop()` 호출 금지

```
#include <stack>
std::stack<int> values;

values.push(10);
values.push(20);
values.push(30);

std::cout << values.top() << '\n'; // 30
values.pop();
std::cout << values.top() << '\n'; // 20
```



- `std::stack`은 컨테이너 어댑터
- 기본 기반 컨테이너는 `std::deque`
- 기반 컨테이너 요구 기능 → `back()`, `push_back()`, `pop_back()`
- `std::deque`, `std::vector`, `std::list` 사용 가능
- 반복자 제공 안 함
- 중간 원소 접근 제한 → LIFO 규칙 유지

```
std::stack<int> defaultStack;  
std::stack<int, std::vector<int>> vectorStack;  
std::stack<int, std::list<int>> listSta;
```

```
struct Command  
{  
    std::string name; int value;  
    Command(std::string name, int value)  
        : name(std::move(name)), value(value)  
    {}  
};
```

```
std::stack<Command> commands;  
commands.emplace("Move", 10);
```



● 대표 활용

- ◆ 실행 취소
- ◆ 괄호 검사
- ◆ 깊이 우선 처리
- ◆ 재귀 호출을 반복 구조로 변환
- ◆ 최근 상태부터 역순 처리

입력 A → 입력 B → 입력 C
↓
top()
↓
C

```
std::stack<std::string> undoHistory;
```

```
undoHistory.push("Create object");
```

```
undoHistory.push("Move object");
```

```
undoHistory.push("Delete object");
```

```
// 실행 취소
```

```
while (!undoHistory.empty())
```

```
{
```

```
    std::cout << "undo: "
```

```
        << undoHistory.top()
```

```
        << '\n';
```

```
    undoHistory.pop();
```

```
}
```



- 컨테이너 어댑터
- 선입선출(FIFO) 처리
- 기본 기반 컨테이너 → `std::deque`



- 먼저 넣은 원소 → 가장 먼저 처리
- `push()` → 뒤에 추가
- `front()` → 앞 원소 확인
- `back()` → 마지막 원소 확인
- `pop()` → 앞 원소 제거
- 빈 상태에서 `front()`, `back()`, `pop()` 호출 금지

```
#include <queue>
std::queue<std::string> messages;

messages.push("Connect");
messages.push("Load");
messages.push("Start");

std::cout << messages.front() << '\n';
messages.pop();
std::cout << messages.front() << '\n';
```



- 기반 컨테이너 요구 기능 → `front()`, `back()`, `push_back()`, `pop_front()`
- `std::deque`, `std::list` 사용 가능
- `std::vector` → `pop_front()`가 없어 사용 불가
- 반복자 제공 안 함
- 중간 원소 접근 제한 → FIFO 규칙 유지

```
std::queue<int> defaultQueue;  
std::queue<int, std::list<int>> listQueue;  
  
std::queue<Command> commands;  
commands.emplace("Attack", 30);
```





- 작업 요청 순서대로 처리
- 네트워크 메시지 처리
- 이벤트 처리
- 너비 우선 탐색(BFS)
- front()의 참조 → pop() 이후 사용 금지

```
struct Job
{
    int id;
    std::string description;
};
```

```
std::queue<Job> jobs;
jobs.push({1, "Load texture"});
jobs.push({2, "Create mesh"});
jobs.push({3, "Save scene"});
```

```
while (!jobs.empty())
{
    Job job = std::move(jobs.front());
    jobs.pop();

    std::cout << job.id
                << ": "
                << job.description
                << '\n';
}
```



- 컨테이너 어댑터
- 삽입 순서가 아닌 우선순위 기준 처리
- 기본 기반 컨테이너 → `std::vector`



- 기본 `std::priority_queue<int>` → 가장 큰 값이 `top()`
- `push()` → 원소 추가
- `top()` → 최고 우선순위 원소 확인
- `pop()` → 최고 우선순위 원소 제거

```
#include <queue>
```

```
std::priority_queue<int> values;
```

```
values.push(30);
```

```
values.push(10);
```

```
values.push(50);
```

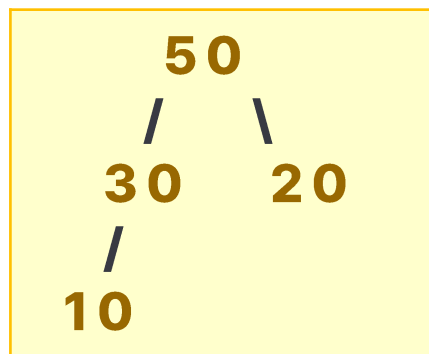
```
values.push(20);
```

```
std::cout << values.top() << '\n'; // 50
```



12.7.2 힙 구조와 top

- 기본 기반 컨테이너: `std::vector`
- 기반 컨테이너 → 힙(heap) 조건 유지
- 전체 원소가 정렬된 구조는 아님
- 최고 우선순위 원소 → `top()`에 유지
- 기반 컨테이너 → 임의 접근 반복자, `front()`, `push_back()`, `pop_back()` 필요



연산	시간 복잡도
<code>top()</code>	$O(1)$
<code>push()</code> , <code>emplace()</code>	$O(\log n)$
<code>pop()</code>	$O(\log n)$

```
std::priority_queue<int, std::deque<int>> values;
```



- 사용자 정의 타입 → 비교 함수 객체로 우선순위 지정
- 비교 함수가 true를 반환한 왼쪽 원소 → 상대적으로 낮은 우선순위
- 동점 처리 필요 → 보조 기준 추가

```
struct Task
```

```
{  
    int id, priority;  
};
```

```
struct CompareTask
```

```
{  
    bool operator()(const Task& left, const Task& right) const  
    {  
        // true를 반환한 left가 더 낮은 우선순위  
        // 큰 priority가 먼저 처리  
        return left.priority < right.priority;  
    }  
};
```

```
std::priority_queue<Task, std::vector<Task>, CompareTask> tasks;
```



- 기본 `std::less<T>` → 최대값 우선
- `std::greater<T>` → 최소값 우선

```
std::priority_queue<int> maxQueue;
```

```
std::priority_queue<  
    int,  
    std::vector<int>,  
    std::greater<int>> minQueue;
```



- 정렬 연관 컨테이너 → 순서 규칙 필요
- 비 정렬 연관 컨테이너 → 해시와 동등 비교 필요
- 타입 자체의 자연스러운 순서와 컨테이너별 정책 구분



- 타입 전체에 자연스러운 하나의 순서
→ 비교 연산자 정의 가능
- C++20 $\langle = \rangle$ → 기본 비교 생성 가능
- 기본 비교 → 멤버 선언 순서 기준

```
#include <compare>
```

```
struct ItemKey
```

```
{
```

```
    int category, id;
```

```
    auto operator<=>(const ItemKey&) const = default;  
};
```

```
std::set<ItemKey> keys;
```

```
keys.insert({1, 1002});
```

```
keys.insert({1, 1001});
```

```
keys.insert({2, 1001});
```





- 같은 타입 → 서로 다른 정렬 기준 적용 가능
- 컨테이너별 비교 정책 분리
- 동점 조건 → 보조 기준으로 구분
- 저장 중 비교 기준 변경 → 정렬 규칙 훼손 가능

```
struct Player
{
    int id;
    std::string name;
    int score;
};
```

```
struct CompareByScore
{
    bool operator()(const Player& left, const Player& right) const
    {
        if (left.score != right.score)
        {
            return left.score > right.score;
        }
        return left.id < right.id;
    }
};

std::set<Player, CompareByScore> scoreRanking;
```



- 사용자 정의 키 → 해시 함수 객체 제공 가능
- 키를 구성하는 멤버 → 해시 계산에 반영
- 같은 키 → 일관된 해시값 생성

```
struct PlayerKey
{
    int serverId, playerId;
    bool operator==(const PlayerKey&) const = default;
};

struct PlayerKeyHash
{
    std::size_t operator()(const PlayerKey& key) const noexcept
    {
        std::size_t first = std::hash<int>{}(key.serverId);
        std::size_t second = std::hash<int>{}(key.playerId);
        return first ^ (second << 1);
    }
};
```



- 해시 함수 → 후보 버킷 선택
- 동등 비교 → 최종 키 확인
- 동등 비교와 해시 → 같은 키 정의 공유
- 동등한 키 → 반드시 같은 해시값
- 서로 다른 키의 해시 충돌 → 허용

동등한 키
↓
반드시 같은





- 값 저장 → 컨테이너가 객체 수명 관리
- 원시 포인터 저장 → 객체 소유권과 별개
- 스마트 포인터 저장 → 소유권을 타입으로 표현
- 다형 객체 저장 → 기반 클래스 스마트 포인터 활용



- 컨테이너가 원소 객체의 수명 관리
- 원소 제거 → 해당 객체 소멸
- 컨테이너 소멸 → 남은 원소 함께 소멸
- 소유권이 명확하고 관리가 단순

```
std::map<int, Item> items;  
items.insert({1001, Item{"Potion", 50}});  
items.erase(1001);
```





- 컨테이너 → 포인터 값만 저장
- `erase()` → 가리키는 객체 자동 소멸 안 함
- 객체의 실제 수명 → 별도 관리
- 동적 객체 소유 필요 → 스마트 포인터 고려

```
std::map<int, Item*> items;  
Item potion{"Potion", 50};  
items.insert({1001, &potion});
```





- 컨테이너 → 객체의 단독 소유권 보관
- unique_ptr → 복사 불가, 이동 가능
- 원소 제거 → 객체 자동 삭제
- 컨테이너 소멸 → 남은 객체 자동 삭제
- try_emplace() → 키 존재 시, 이동 인수 소비 안함

```
#include <memory>
```

```
std::map<int, std::unique_ptr<Item>> items;  
items.try_emplace(1001,  
std::make_unique<Item>(Item{"Potion", 50}));
```

```
auto item = std::make_unique<Item>(Item{"Ether", 80});  
items.insert({1002, std::move(item)});
```



- 여러 위치의 공동 소유가 필요할 때 사용
- 컨테이너에서 제거되어도 다른 shared_ptr 존재 → 객체 유지
- 참조 계수 관리 비용 존재
- 순환 소유 주의
- 단독 소유 가능 → unique_ptr 우선 고려

```
std::map<int, std::shared_ptr<Item>> items;  
auto potion = std::make_shared<Item>(Item{"Potion", 50});  
items.insert({1001, potion});  
  
items.erase(1001);  
std::cout << potion->name << '\n';
```





- 서로 다른 파생 객체 → 기반 클래스 스마트 포인터로 저장
- 기반 클래스 → 가상 소멸자 필요
- 기반 타입 값 저장 → 객체 잘림 발생 가능
- 스마트 포인터 + 가상 함수 → 다형성 유지

```
class GameObject
{
public:
    virtual ~GameObject() = default;
    virtual void Update() = 0;
};
```

```
class Player : public GameObject {
public:
    void Update() override
    {
        std::cout << "Player update\n";
    }
};

class Monster : public GameObject {
public:
    void Update() override
    {
        std::cout << "Monster update\n";
    }
};
```

```
std::map<int, std::unique_ptr<GameObject>> objects;
objects.try_emplace(1, std::make_unique<Player>());
objects.try_emplace(2, std::make_unique<Monster>());
```



12.9.6 반복자·포인터·참조의 유효성과 무효화

컨테이너와 연산	반복자	포인터와 참조
set, map 계열 삽입	유지	유지
set, map 계열 제거	제거된 원소만 무효	제거된 원소만 무효
unordered 삽입, 재 해시 없음	유지	유지
unordered 삽입, 재 해시 발생	전체 무효	유지
unordered reserve(), rehash()	재 해시 시 전체 무효	유지
unordered 제거	제거된 원소만 무효	제거된 원소만 무효
clear()	전체 무효	전체 무효
컨테이너 소멸	전체 무효	전체 무효

```
for (auto iter = items.begin(); iter != items.end(); )  
{  
    if (iter->second.empty())  
        iter = items.erase(iter);  
    else  
        ++iter;  
}
```

- 제거 중 순회에서는
erase()가 반환한 반복자 사용



요구 사항	적합한 타입
중복 없는 정렬 키	<code>std::set</code>
중복을 허용하는 정렬 키	<code>std::multiset</code>
정렬된 키와 값	<code>std::map</code>
같은 정렬 키에 여러 값	<code>std::multimap</code>
중복 없는 빠른 키 검색	<code>std::unordered_set</code>
빠른 키와 값 검색	<code>std::unordered_map</code>
같은 해시 키에 여러 원소	<code>unordered_multi</code> 계열
최근 원소부터 처리	<code>std::stack</code>
도착 순서대로 처리	<code>std::queue</code>
우선순위 높은 원소부터 처리	<code>std::priority_queue</code>

- 키 정렬 여부 확인
- 중복 허용 여부 확인
- 범위 검색 필요 여부 확인
- 평균·최악 탐색 비용 확인
- 반복자 안정성 확인
- 객체 소유권 확인
- 실제 성능 중요
 - 대상 데이터와 연산 패턴으로 측정





- 연관 컨테이너 → 키 기준 저장·탐색
- set → 키만 저장, map → 키와 값 저장
- multi 계열 → 동등한 키 여러 개 허용
- 정렬 연관 컨테이너 → 비교 함수로 키 순서 결정
- 비 정렬 연관 컨테이너 → 해시값으로 버킷 선택
- `map::operator[]` → 키가 없으면 기본값 삽입
- `try_emplace()` → 키가 없을 때만 값 생성
- `insert_or_assign()` → 삽입 또는 기존 값 갱신



- stack → 후입선출, queue → 선입선출
- priority_queue → 우선순위 기반 처리
- 값 객체 저장 → 컨테이너가 수명 관리
- 원시 포인터 저장 → 객체 소유권 자동 관리 안 함
- unique_ptr → 단독 소유, shared_ptr → 공동 소유
- 정렬 연관 컨테이너 삽입 → 기존 반복자와 참조 유지
- 비 정렬 연관 컨테이너 재 해시 → 반복자 무효화
- 컨테이너 선택 → 데이터의 키 관계와 처리 규칙 기준

